Assignment Number:19.3

K.Akshay

2303A51817

Q.No.	Question	*Expected Time to complete*
1	**Lab 19 – Code Translation: Converting Between Programming Languages** **Lab Objectives:** • Understand how AI tools can assist in translating code between different programming languages. • Learn to verify correctness and functionality after translation. • Explore syntactic and semantic differences between languages (e.g., Python, Java, C++). • Practice debugging and optimizing AI-translated code.	Week10 - Wednesday

	Task 1 – Python to C++ Conversion
	Task:
	Translate a simple Python class into C++ using AI assistance.
	Instructions:
	• Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
	• Ensure proper handling of:
	  o Constructors
	  o Data types
	  o Access specifiers
	• Compile and run both versions to verify output consistency
	Expected Output:
	An equivalent working C++ class translated from Python.
	Prompt :
	Translate this Python class into C++ with proper constructor, private attributes, and a public method. Ensure the output matches the Python version. Python: class Student: def __init__(self, name, age, grade): self.name = name self.age = age self.grade = grade def display_info(self): print(f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}")
	Code :	

```
print('```cpp')
print('// Student.h (Header file for the Student class)')
print('''class Student {
private:
    std::string name;
    int age;
    char grade;

public:
    // Constructor
    Student(std::string n, int a, char g) {
        name = n;
        age = a;
        grade = g;
    }

    // Public method to display student information
    void display_info() {
        std::cout << "Name: " << name << ", Age: " << age << ", Grade: " << grade << std::endl;
    }
};
''')
print('// main.cpp (Example usage)')
print('''#include <iostream>''')
print('''#include <string>''')
print('''#include "Student.h" // Include the header file''')
print('''int main() {
```

```
    // Create a Student object
    Student student1("Alice", 20, 'A');

    // Call the display_info method
    student1.display_info();

    // You can create more students
    Student student2("Bob", 22, 'B');
    student2.display_info();

    return 0;
}
''')
print('```')
```

```cpp
// Student.h (Header file for the Student class)
class Student {
private:
    std::string name;
    int age;
    char grade;

public:
    // Constructor
    Student(std::string n, int a, char g) {
        name = n;
        age = a;
        grade = g;
    }

    // Public method to display student information
    void display_info() {
        std::cout << "Name: " << name << ", Age: " << age << ", Grade: " << grade << std::endl;
    }
};

// main.cpp (Example usage)
#include <iostream>
#include <string>
#include "Student.h" // Include the header file
int main() {
    // Create a Student object
    Student student1("Alice", 20, 'A');
```

```
    // Call the display_info method
    student1.display_info();

    // You can create more students
    Student student2("Bob", 22, 'B');
    student2.display_info();

    return 0;
}

```
```
```

```
%%writefile Student.h
#include <string>
#include <iostream>

class Student {
private:
    std::string name;
    int age;
    char grade;

public:
    Student(std::string n, int a, char g) {
        name = n;
        age = a;
        grade = g;
    }

    void display_info() {
        std::cout << "Name: " << name << ", Age: " << age << ", Grade: " << grade << std::endl
    }
};
```

```
Writing Student.h
```

```
[7]
%%writefile main.cpp
#include <iostream>
#include "Student.h" // Include the header file

int main() {
    Student student1("Alice", 20, 'A');
    student1.display_info();

    Student student2("Bob", 22, 'B');
    student2.display_info();

    return 0;
}

Writing main.cpp
```

Step 3: Compile the C++ code

Use the `g++` compiler to compile your `.cpp` file into an executable. The `-o` flag specifies the output file name.

```
[8]
!g++ main.cpp -o main
```

Step 4: Run the executable

Once compiled, you can run the executable directly using `./` followed by the executable name.

```
[9]
!./main

Name: Alice, Age: 20, Grade: A
Name: Bob, Age: 22, Grade: B
```

Explanation : Translated your Python Student class into a C++ equivalent, including a header file (Student.h) for the class definition and a main.cpp file for an example of how to use it.

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

Prompt :

Convert this Java isPrime() method into a Python function. Ensure Pythonic syntax and logic, then test with multiple inputs. Java: public static boolean isPrime(int n) { if (n <= 1) return false; for (int i = 2; i <= Math.sqrt(n); i++) { if (n % i == 0) return false; } return true; } Expected Output: A correct Python function equivalent to isPrime() that works for various inputs.

Code :

```python
import math

def is_prime(n):
    """
    Checks if a given integer n is a prime number.
    Equivalent to the provided Java isPrime method.
    """
    if n <= 1:
        return False
    # Check for divisibility from 2 up to the square root of n
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

print("Python is_prime() function created successfully.")
```

```
Python is_prime() function created successfully.
```

```python
# Test the is_prime() function with multiple inputs
print(f"Is 0 prime? {is_prime(0)}")       # Expected: False
print(f"Is 1 prime? {is_prime(1)}")       # Expected: False
print(f"Is 2 prime? {is_prime(2)}")       # Expected: True
print(f"Is 3 prime? {is_prime(3)}")       # Expected: True
print(f"Is 4 prime? {is_prime(4)}")       # Expected: False (4 % 2 == 0)
print(f"Is 7 prime? {is_prime(7)}")       # Expected: True
print(f"Is 9 prime? {is_prime(9)}")       # Expected: False (9 % 3 == 0)
print(f"Is 11 prime? {is_prime(11)}")     # Expected: True
print(f"Is 25 prime? {is_prime(25)}")     # Expected: False (25 % 5 == 0)
print(f"Is 29 prime? {is_prime(29)}")     # Expected: True
print(f"Is 97 prime? {is_prime(97)}")     # Expected: True
```

```
Is 0 prime? False
Is 1 prime? False
Is 2 prime? True
Is 3 prime? True
Is 4 prime? False
Is 7 prime? True
Is 9 prime? False
Is 11 prime? True
Is 25 prime? False
Is 29 prime? True
Is 97 prime? True
```

Explanation : The is_prime() function checks if a given integer is a prime number. Here's a breakdown of its logic:A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.Initial Check (if n <= 1):The function first checks if the input number n is less than or equal to 1. By definition, numbers less than or equal to 1 are not prime. So, it immediately returns False for these cases.

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Prompt :

Convert this pseudocode for bubble sort into Python. Ensure Pythonic syntax and test with sample input lists. Pseudocode: procedure bubbleSort(A) n = length(A) for i = 0 to n-1 for j = 0 to n-i-2 if A[j] > A[j+1] swap A[j], A[j+1] Expected Output: A working Python program that sorts lists correctly.

Code :

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

print("bubble_sort() function created successfully.")
```

```
bubble_sort() function created successfully.
```

```python
# Test the bubble_sort function with a sample list
sample_list_1 = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {sample_list_1}")
sorted_list_1 = bubble_sort(sample_list_1)
print(f"Sorted list: {sorted_list_1}")

sample_list_2 = [5, 1, 4, 2, 8]
print(f"Original list: {sample_list_2}")
sorted_list_2 = bubble_sort(sample_list_2)
print(f"Sorted list: {sorted_list_2}")
```

```python
sample_list_3 = []
print(f"Original list: {sample_list_3}")
sorted_list_3 = bubble_sort(sample_list_3)
print(f"Sorted list: {sorted_list_3}")

sample_list_4 = [10]
print(f"Original list: {sample_list_4}")
sorted_list_4 = bubble_sort(sample_list_4)
print(f"Sorted list: {sorted_list_4}")

sample_list_5 = [5, 5, 5, 5]
print(f"Original list: {sample_list_5}")
sorted_list_5 = bubble_sort(sample_list_5)
print(f"Sorted list: {sorted_list_5}")
```

```
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Original list: [5, 1, 4, 2, 8]
Sorted list: [1, 2, 4, 5, 8]
Original list: []
Sorted list: []
Original list: [10]
Sorted list: [10]
Original list: [5, 5, 5, 5]
Sorted list: [5, 5, 5, 5]
```

Explanation : The bubble_sort() function was implemented and tested with different lists, including empty, single-element, and duplicate-element lists, all with correct results.

Task 4 – SQL to Pandas Query

Task:

Translate a **SQL query** into a **Pandas DataFrame operation** in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Prompt :

Convert this SQL query into an equivalent Pandas DataFrame operation in Python. Test the code on a sample DataFrame to ensure correctness. SQL: SELECT name, salary FROM employees WHERE salary > 50000; Expected Output: A Pandas query that retrieves the correct subset of data.

Code :

[14]
✓ 0s

```python
import pandas as pd

# Create a sample DataFrame simulating the 'employees' table
employees_df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'age': [24, 30, 29, 35, 22],
    'salary': [60000, 45000, 75000, 50000, 80000]
})

print("Original DataFrame:")
display(employees_df)
```

```
Original DataFrame:
```

	name	age	salary
0	Alice	24	60000
1	Bob	30	45000
2	Charlie	29	75000
3	David	35	50000
4	Eve	22	80000

Next steps: Generate code with employees_df | New interactive sheet

```python
# Equivalent Pandas operation for: SELECT name, salary FROM employees WHERE salary > 50

high_earners = employees_df[employees_df['salary'] > 50000][['name', 'salary']]

print("Employees with salary > 50000:")
display(high_earners)
```

```
Employees with salary > 50000:
```

	name	salary
0	Alice	60000
2	Charlie	75000
4	Eve	80000

Next steps: Generate code with high_earners | New interactive sheet

Explanation : This part performs the WHERE salary > 50000 clause. It creates a boolean Series where True indicates rows where the salary is greater than 50000.When this boolean Series is used to index employees_df, it effectively filters the DataFrame, keeping only the rows that satisfy the condition.

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
  - Syntax differences
  - Library support
  - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

Prompt :

```
Translate this algorithm from Python into C. Ensure equivalent logic, correct syntax, and test both versions for consistency. Document translation challenges such as syntax differences, library support, and memory management. Python: def linear_search(arr, target): for i in range(len(arr)): if arr[i] == target: return i return -1 Expected Output: 1. Equivalent C implementation. 2. Verified correctness with sample inputs. 3. Notes on translation challenges.
```

Code :

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

print("Python linear_search() function created successfully.")
```

```
Python linear_search() function created successfully.
```

```c
%%writefile linear_search.c
#include <stdio.h>

// C implementation of linear search
int linear_search(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Target found, return its index
        }
    }
    return -1; // Target not found
}

int main() {
    // Test cases
    int arr1[] = {10, 20, 30, 40, 50};
    int size1 = sizeof(arr1) / sizeof(arr1[0]);
    int target1 = 30;
    printf("C: Array: {10, 20, 30, 40, 50}, Target: %d, Result: %d\n", target1, linear_search(arr1, size1, target1)); // Expected: 2

    int target2 = 60;
    printf("C: Array: {10, 20, 30, 40, 50}, Target: %d, Result: %d\n", target2, linear_search(arr1, size1, target2)); // Expected: -1

    int arr2[] = {}; // Empty array (size 0)
    int size2 = 0;
    int target3 = 5;
    printf("C: Array: {}, Target: %d, Result: %d\n", target3, linear_search(arr2, size2, target3)); // Expected: -1

    int arr3[] = {5};
    int size3 = sizeof(arr3) / sizeof(arr3[0]);
    int target4 = 5;
    printf("C: Array: {5}, Target: %d, Result: %d\n", target4, linear_search(arr3, size3, target4)); // Expected: 0

    return 0;
}
```

```
Writing linear_search.c
```

```python
# Compile the C code
!gcc linear_search.c -o c_linear_search
print("C code compiled successfully.")
```

```
linear_search.c: In function ‘main’:
linear_search.c:26:5: warning: ‘linear_search’ accessing 4 bytes in a region of size 0 [-Wstringop-overflow=]
   26 |     printf("C: Array: {}, Target: %d, Result: %d\n", target3, linear_search(arr2, size2, target3)); // Expected: -1
      |     ^~~~
```

```python
arr_py3 = [5]
target_py4 = 5
print(f"Python: Array: {arr_py3}, Target: {target_py4}, Result: {linear_search(arr_py3, target_py4)}") # Expected: 0
```

```
Python: Array: [10, 20, 30, 40, 50], Target: 30, Result: 2
Python: Array: [10, 20, 30, 40, 50], Target: 60, Result: -1
Python: Array: [], Target: 5, Result: -1
Python: Array: [5], Target: 5, Result: 0
```

Explanation : For this simple linear_search, explicit memory management (like malloc or free) isn't strictly required as we're using stack-allocated arrays. However, for more complex C programs involving dynamic data structures or larger arrays, manual memory allocation and deallocation become crucial, which Python handles automatically via its garbage collector.